Super učni vodič APS2 v2

Super učni vodič APS2 - popravljena cram-ready verzija

Ta verzija je napisana tako, da lahko iz nje dejansko ponavljaš snov, ne samo bereš odgovore. Za vsak sklop glej: kaj je ideja, kdaj jo uporabiš, postopek, mini primer, tipične napake in preverjanje.

0. Kako se učiti iz tega vodiča

Najprej preleti A4 povzetek, potem ta vodič. Pri vsaki metodi naredi vsaj en ročni primer. APS2 naloge skoraj vedno ocenjujejo postopek: če napišeš samo rezultat, izgubiš točke tudi, ko je rezultat pravilen.

Red učenja:

1. Asimptotika + amortizacija.
2. Osnovne podatkovne strukture: tabele, slovarji, seznami, kopice.
3. Drevesa: BST, AVL, rdeče-črna, B/B+.
4. Zgoščevanje, Bloom, disjunktne množice.
5. DP: stopnice/Fibonacci, nahrbtnik, verižno množenje.
6. Grafi: samo Prim, Kruskal, topološko, Dijkstra, Bellman-Ford, Welsh-Powell.
7. P/NP in redukcije.

1. Asimptotična zahtevnost

Kaj moraš znati

Znati moraš prepoznati red rasti algoritma in ga povezati z realno velikostjo podatkov. Na izpitu se pogosto pojavi vprašanje tipa: "Koliko časa traja algoritem, če je n = ... in procesor naredi ... operacij/s?"

Definicije

- `O(f(n))`: algoritem ne raste hitreje kot `f(n)` do konstante. Zgornja meja.
- `Ω(f(n))`: algoritem raste vsaj tako hitro kot `f(n)`. Spodnja meja.
- `Θ(f(n))`: oboje hkrati. Tesna meja.

Primer: `3n^2 + 10n + 5 = Θ(n^2)`.

Kako oceniš zanke

```
for i = 1..n:
    for j = 1..n:
        O(1)
```

`n*n` ponovitev -> `Θ(n^2)`.

```
for i = 1..n:
    for j = 1..i:
        O(1)
```

`1+2+...+n = n(n+1)/2 = Θ(n^2)`.

```
i = 1
while i < n:
    i = 2*i
```

Število ponovitev je `log2 n`, zato `Θ(log n)`.

Rekurzije

Tipične oblike:

- `T(n)=T(n/2)+O(1)` -> `O(log n)` (binarno iskanje).
- `T(n)=2T(n/2)+O(n)` -> `O(n log n)` (merge sort).
- `T(n)=T(n-1)+O(1)` -> `O(n)`.
- `T(n)=T(n-1)+T(n-2)+O(1)` -> eksponentno, če ne shranjuješ rezultatov.

Ocena realnega časa

Postopek:

1. Dobi `f(n)`.
2. Če je dana konstanta `C`, računaj `C*f(n)`.
3. Deli s številom operacij/s.

Mini primer: `n=10^7`, `O(n^2)`, `C=1`, 1 GHz.

`n^2 = 10^14`; `10^14 / 10^9 = 10^5 s ≈ 27.8 h`.

Tipične napake

- Pri polnem grafu pozabiš, da ima `Θ(n^2)` povezav.
- Zamenjaš `log n` in `n log n`.
- Iz tega, da poznaš počasen algoritem, sklepaš, da je problem težak. To ne drži.

2. ADT in osnovne podatkovne strukture

ADT vs podatkovna struktura

ADT pove, katere operacije obstajajo in kaj pomenijo. Podatkovna struktura pove, kako je to implementirano.

Primer: ADT slovar ima `insert`, `find`, `delete`. Implementacija je lahko seznam, BST, zgoščena tabela, B-drevo ...

Tabela

Tabela je dobra, ko poznaš indeks.

- `get(i)`: `O(1)`
- `put(i,x)`: `O(1)`
- vstavljanje na poljubno mesto: lahko `O(n)`, ker premikaš elemente

Dinamična tabela

Problem: navadna tabela ima fiksno kapaciteto. Rešitev: ko je polna, naredimo večjo in prekopiramo elemente.

Najbolj smiselno: podvajanje velikosti.

Zakaj?

- povečanje za +1: kopiranje $1+2+...+n = \Theta(n^2)$;
- podvajanje: skupno kopiranje manj kot $2n$, zato `insert` amortizirano $O(1)$.

Mini primer: kapacitete $1, 2, 4, 8, 16$. Do 16 elementov skupno prekopiraš $1+2+4+8=15$ elementov.

Tabela tabel

Struktura hrani elemente v urejenih tabelah $A_0, A_1, \ldots$, kjer je velikost $A_i = 2^i$.

Ideja: zasedene tabele ustrezajo bitom v binarnem zapisu števila elementov.

Primer: $n=13 = 8+4+1 = 1101_2$, zato so zasedene A_3, A_2, A_0.

Iskanje:

1. Pregledaš vse zasedene tabele.
2. V vsaki narediš binarno iskanje.
3. Tabel je $O(\log n)$, iskanje v posamezni je $O(\log n)$.
4. Skupaj $O(\log^2 n)$.

Vstavljanje:

1. Narediš tabelo velikosti 1 z novim elementom.
2. Če je A_0 prazna, jo postaviš tja.
3. Če je polna, zlij tabeli in prenos nesi v A_1.
4. Ponavljaš kot binarni števec.

Tipična napaka: pri vprašanju “koliko tabel potrebujemo” loči med številom zasedenih tabel in številom vseh nivojev do največjega.

3. Seznami in preskočni seznami

Navaden povezan seznam

Iskanje je $O(n)$, tudi če je seznam urejen, ker ne moreš skočiti na sredino kot pri tabeli.

Vstavljanje na začetek je $O(1)$, vstavljanje na pravo urejeno mesto pa zahteva iskanje mesta.

Popolnoma dopolnjen seznam

Dodamo kazalce $+1, +2, +4, +8, \ldots$.

Iskanje je podobno binarnemu zapisu: z največjim skokom določiš polovico, z naslednjim četrtino ...

Iskanje: $O(\log n)$.

Problem: vstavljanje in brisanje lahko zahtevata popravljanje ogromno kazalcev -> $O(n)$.

Preskočni seznam

Preskočni seznam reši problem s slučajno višino elementov.

Pri vstavljanju:

1. Element je vedno na nivoju 0.
2. Mečeš kovanec.
3. Dokler pada "uspeh", element dobi še en nivo.
4. Pri verjetnosti 1/2 ima malo elementov veliko višino.

Iskanje:

1. Začneš zgoraj levo.
2. Premikaš se desno, dokler naslednji element ni prevelik.
3. Potem greš dol.
4. Na dnu najdeš element ali mesto, kjer bi moral biti.

Pričakovano:

- iskanje $O(\log n)$;
- vstavljanje $O(\log n)$;
- prostor $O(n)$.

Mini primer z višinami:

```
nivo 2: 7 -------- 8
nivo 1: 6 -- 7 -- 8 -- 9
nivo 0: 3 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
```

Pazi: $O(\log n)$ je tudi $O(n)$, ampak ni $\Omega(n^2)$.

4. Slovarji, zgoščene tabele in Bloomov filter

Slovar

Slovar hrani pare $(ključ, vrednost)$.

Če vstaviš isti ključ dvakrat, se v običajni definiciji vrednost posodobi. Če naloga eksplicitno obravnava duplikate kot elemente, potem jih hraniš po dogovoru naloge.

Zgoščena tabela

Ideja: ključ k preslikamo v indeks $h(k)$.

Če je m velikost tabele:

- metoda deljenja: $h(k)=k \bmod m$;
- metoda množenja: $h(k)=(k*p) \bmod m$.

Dobro je, da je m ali p izbran tako, da razbije vzorce v ključih.

Sovpadanje

Sovpadanje pomeni, da dva ključa padeta v isto celico.

Veriženje

V vsaki celici je koš: seznam, BST ali druga struktura.

Operacije:

1. Izračunaj `h(k)`.
2. Pojdi v koš.
3. Tam išči/vstavi/briši.

Pričakovani čas: $O(1+\alpha)$, kjer je $\alpha=n/m$.

Odprto naslavljanje

Vse je v isti tabeli. Če je celica polna, poskusiš naslednjo.

Linearno:

$h(k,i) = (h(k)+i) \bmod m$.

Postopek vstavljanja:

1. $i=0$.
2. Poglej $h(k,i)$.
3. Če je prosto, vstavi.
4. Sicer povečaj i.

Pazi:

• brisanje zahteva oznako `DELETED`;
• pri visoki zasedenosti število poskusov močno naraste;
• linearno naslavljanje povzroča gruče.

Bloomov filter

Bloomov filter ima bitno tabelo in k zgoščevalnih funkcij.

Vstavljanje:

1. Za element izračunaš $h_1(x),\ldots,h_k(x)$.
2. Ustrezne bite nastaviš na 1.

Preverjanje:

1. Izračunaš iste bite.
2. Če je vsaj eden 0 -> zagotovo NE.
3. Če so vsi 1 -> verjetno DA.

False positive je možen, false negative ne, dokler ne brišeš.

Brisanje:

- navaden Bloomov filter ne zna varno brisati;
- counting Bloom filter namesto bitov hrani števce.

5. Kopice in prioritetne vrste

Prioritetna vrsta

ADT z operacijami:

- `min` ali `max`;
- `insert`;
- `deleteMin` ali `deleteMax`.

Min-kopica

Lastnost: vsak starš je manjši ali enak otrokom.

Ni BST. Levo/desno poddrevo nista urejeni po BST pravilu.

Operacije:

- `min`: `O(1)`, koren;
- `insert`: dodaš na konec in popravljaš navzgor, `O(log n)`;
- `deleteMin`: koren zamenjaš z zadnjim, pobrišeš zadnjega, popravljaš navzdol, `O(log n)`;
- `heapify` celotne tabele: `O(n)`.

Pasti:

- največji element v min-kopici je nekje med listi, ne nujno na skrajnem desnem mestu;
- urejen izpis vseh elementov z zaporednim `deleteMin` traja `O(n log n)`.

6. Drevesa

Dvojiško iskalno drevo (BST)

Pravilo:

- levo poddrevo ima manjše ključe;
- desno poddrevo ima večje ključe.

Iskanje, vstavljanje, brisanje: `O(h)`.

Minimum: pojdi čim bolj levo. Maksimum: čim bolj desno.

Če je drevo uravnoteženo, `h=O(log n)`. Če vstaviš že urejene elemente v navaden BST, lahko dobiš seznam in `h=O(n)`.

AVL drevo

AVL je BST z dodatnim pogojem:

`|height(left)-height(right)| <= 1` za vsako vozlišče.

Po vstavljanju:

1. Vstavi kot v BST.
2. Hodi nazaj proti korenu.
3. Najdi prvo neuravnoteženo vozlišče.
4. Popravi z rotacijo.

Rotacije:

• LL: desna rotacija.
• RR: leva rotacija.
• LR: leva rotacija pri levem otroku, nato desna.
• RL: desna rotacija pri desnem otroku, nato leva.

Čas: vse osnovne operacije `O(log n)`.

Rdeče-črno drevo

Je BST z barvami.

Pravila:

1. Vozlišče je rdeče ali črno.
2. Koren je črn.
3. Prazni listi NIL so črni.
4. Rdeče vozlišče nima rdečega otroka.
5. Vsaka pot od vozlišča do NIL lista ima enako število črnih vozlišč.

Vstavljanje:

1. Vstavi kot BST.
2. Novo vozlišče pobarvaj rdeče.
3. Če je starš črn, konec.
4. Če je starš rdeč, glej strica.
5. Rdeč stric -> prebarvanje.
6. Črn stric -> rotacija + prebarvanje.

AVL vs RČ:

• AVL je bolj uravnotežen -> hitrejše iskanje.
• RČ ima manj popravljanja -> dobro za veliko vstavljanj/brisanj.

B-drevo in B+ drevo

Uporaba: baze in diski. Cilj je malo dostopov do diska, zato ima vozlišče več ključev.

B-drevo:

- ključi so v notranjih vozliščih in listih;
- vsi listi so na isti globini;
- ob prepolnem vozlišču se razcepi.

B+ drevo:

- podatki so v listih;
- notranja vozlišča so indeks;
- listi so pogosto povezani za hitro zaporedno branje.

7. Disjunktne množice

Cilj

Vzdržujemo razbitje elementov na disjunktne množice.

Operacije:

- `makeSet(x)`: naredi množico `{x}`;
- `find(x)`: vrne predstavnika množice;
- `union(x,y)`: združi množici.

Učinkovita izvedba

Gozd dreves, kjer vsak otrok kaže na starša, koren pa je predstavnik.

Optimizaciji:

- union by rank/size: manjše drevo pripni pod večje;
- path compression: pri `find` vse elemente na poti pripni direktno na koren.

Amortizirano skoraj konstantno: $O(\alpha(n))$.

Kdaj uporabiš

Kruskalov algoritem: preveriš, ali sta krajišči povezave že v isti komponenti.

8. Amortizirana analiza

Ideja

Worst-case ene operacije je lahko drag, ampak to se ne zgodi pogosto. Amortizirana analiza ocenjuje povprečno ceno v najslabšem zaporedju operacij.

Metoda vsote

Sešteješ realne cene vseh operacij in deliš s številom operacij.

Primer sklad z `multipop(k)`:

- vsak element je enkrat potisnjen;
- vsak element je največ enkrat odstranjen;
- skupno delo pri odstranjevanju je največ število vstavljanj;
- zato je amortizirano `O(1)`.

Bankirska metoda

Poceni operacije plačajo malo več in pustijo kredit.

Primer sklad:

- `push` plača 2: 1 za takojšnje vstavljanje, 1 kredit na elementu;
- ko ga kasneje `pop` odstrani, porabi kredit.

Potencialna metoda

Formula:

```
a_i = c_i + Phi(D_i) - Phi(D_{i-1})
```

Izbereš potencial, ki meri “shranjeno energijo” strukture.

Primer sklad: `Phi(D)=število elementov na skladu`.

9. Dinamično programiranje

Kako ga prepoznaš

Naloga ima optimalno rešitev ali število možnosti in se da razbiti na podobne manjše naloge.

Ključni znaki:

- "največja/najmanjša vrednost";
- "na koliko načinov";
- "optimalna postavitev";
- "uporabi postopek iz predavanj".

Standardni postopek

1. Definiraj stanje.
2. Zapiši rekurenco.
3. Zapiši začetne vrednosti.
4. Določi vrstni red računanja.
5. Če naloga zahteva predmete/pot/oklepaje, rekonstruiraj rešitev.

Fibonacci in stopnice

Stopnice s koraki 1 ali 2:

`f(n)=f(n-1)+f(n-2)`.

Zakaj? Zadnji korak je dolg 1 ali 2.

Osnove:

- `f(0)=1`: en način, da ne narediš nič;
- `f(1)=1`.

0/1 nahrbtnik

Stanje:

`dp[i][w]` = najboljša vrednost s prvimi `i` predmeti in kapaciteto `w`.

Prehod:

Če predmeta ne vzamem:

`dp[i-1][w]`.

Če ga vzamem:

`dp[i-1][w-w_i] + v_i`.

Zato:

`dp[i][w]=max(dp[i-1][w], dp[i-1][w-w_i]+v_i)`.

Če `w_i>w`, predmeta ne moreš vzeti.

Preverjanje: končni odgovor je `dp[n][W]`, predmete najdeš z vračanjem nazaj po tabeli.

Delni nahrbtnik

Če smeš predmete rezati, uporabi greedy:

1. Izračunaj `v_i/w_i`.
2. Uredi padajoče.
3. Jemlji cele predmete, zadnjega po potrebi odreži.

Pazi: isti greedy ne velja za 0/1 nahrbtnik.

Verižno množenje matrik

Matrika `M_i` ima dimenzijo `d_{i-1} x d_i`.

Cena množenja `(a x b)*(b x c)` je `a*b*c`.

Stanje:

`c(i,j)` = najmanjša cena produkta `M_i...M_j`.

Rekurenca:

`c(i,j)=min_{i<=k<j}(c(i,k)+c(k+1,j)+d_{i-1}*d_k*d_j)`.

Osnova:

`c(i,i)=0`.

Računaš po diagonalah: najprej dolžina 1, potem 2, 3 ...

Pazi:

- rezultat produkta je isti ne glede na oklepaje;
- cena je lahko zelo različna;
- vrstni red matrik se ne sme poljubno menjati, razen če naloga eksplicitno sprašuje po veljavnem vrstnem redu iz danih matrik.

10. Grafi - samo zahtevane teme

Največji pretok preskoči.

Predstavitev grafa

Matrika:

- prostor `O(n^2)`;
- hitro preverjanje, ali obstaja povezava `O(1)`;
- naštevanje sosedov je `O(n)`.

Seznam sosedov:

- prostor `O(n+m)`;
- naštevanje sosedov je učinkovito;
- preverjanje posamezne povezave je počasnejše, razen če uporabljaš slovarje.

Minimalno vpeto drevo

MST obstaja za povezan neusmerjen utežen graf.

Lastnosti:

- ima `n-1` povezav;
- poveže vsa vozlišča;
- nima cikla;
- če so vse uteži različne, je MST enoličen.

Prim

Uporaba: MST.

Postopek:

1. Začni v izbranem vozlišču.

2. Množica `T` vsebuje že izbrana vozlišča.
3. Med povezavami iz `T` v `V\T` izberi najcenejšo.
4. Dodaj novo vozlišče in povezavo.
5. Ponavljaj do `n-1` povezav.

Čas:

- matrika + linearno iskanje: `O(n^2)`;
- seznam sosedov + min-kopica: `O((n+m)log n)`.

Kruskal

Postopek:

1. Uredi vse povezave po teži.
2. Začni s praznim gozdom.
3. Povezavo dodaj, če krajišči nista že v isti komponenti.
4. Komponente vodi z disjunktnimi množicami.
5. Ustavi se pri `n-1` povezavah.

Čas: `O(m log m)`.

Topološko urejanje

Samo za DAG.

Kahnov algoritem:

1. Izračunaj vhodne stopnje.
2. V vrsto dodaj vozlišča z vhodno stopnjo `0`.
3. Odstrani vozlišče, izpiši ga, zmanjšaj vhodne stopnje naslednikov.
4. Če ne izpišeš vseh vozlišč, graf ima cikel.

Najkrajše poti v DAG

1. Topološko uredi graf.
2. `d[s]=0`, ostalo ∞.
3. V topološkem vrstnem redu sproščaj izhodne povezave.

Čas: `O(n+m)`.

Dijkstra

Uporaba: najkrajše poti, vse uteži nenegativne.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. Vsi so neobiskani.
3. Izberi neobiskano vozlišče z najmanjšim `d`.

4. Označi kot obiskano.
5. Sprosti vse izhodne povezave.

Sprostitev:

Če `d[u]+w(u,v)<d[v]`, potem `d[v]=d[u]+w(u,v)` in `pred[v]=u`.

Pazi: negativna povezava pomeni, da Dijkstra ni veljaven.

Bellman-Ford

Uporaba:

- negativne uteži;
- preverjanje negativnega cikla.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. `n-1` krat sprosti vse povezave.
3. Še enkrat sprosti vse povezave.
4. Če se kaj izboljša, obstaja dosegljiv negativen cikel.

Čas: `O(nm)`.

Welsh-Powell

Uporaba: hitro barvanje grafa.

Postopek:

1. Vozlišča uredi padajoče po stopnji.
2. Prvemu nepobarvanemu dodeli novo barvo.
3. Z isto barvo pobarvaj vsa naslednja vozlišča, ki niso sosednja nobenemu že pobarvanemu s to barvo.
4. Ponavljaj.

Pazi: rezultat ni nujno optimalen.

11. Kompleksnost P/NP

Odločitveni in optimizacijski problemi

Odločitveni problem ima odgovor DA/NE.

Optimizacijski problem išče najboljšo rešitev po neki kriterijski funkciji.

Pogosto lahko optimizacijski problem rešuješ z več odločitvenimi vprašanji.

P

Odločitveni problemi, ki jih znamo rešiti v polinomskem času.

NP

Odločitveni problemi, kjer znamo pravilnost dane rešitve preveriti v polinomskem času.

Velja $P \subseteq NP$.

Redukcije

Če problem A polinomsko prevedemo na problem B, potem je B vsaj tako težak kot A.

Trik za smer: lažji problem lahko rešujemo s težjim. Če znaš rešiti težjega, znaš rešiti tudi lažjega po prevedbi.

NP-poln

Problem je NP-poln, če:

1. je v NP;
2. vsak problem iz NP se nanj polinomsko reducira.

Če najdeš polinomski algoritem za en NP-poln problem, potem P=NP.

NP-težak

Vsaj tako težak kot NP-polni problemi. Ni nujno odločitveni in ni nujno v NP.

Praktični pristopi

Za NP-težke/NP-polne probleme pogosto uporabljamo:

- hevristike;
- aproksimacijske algoritme;
- lokalno optimizacijo;
- backtracking;
- eksponentne algoritme za majhne vhode.

12. Kako pisati rešitve na izpitu

DA/NE

Oblika:

```
NE. Razlog: ...
Protiprimer: ...
```

Če je trditev formalno ohlapno resnična, zapiši tudi tesnejšo mejo. Primer: $O(\log n)$ je tudi $O(n^2)$, ampak prava tesna meja je $\Theta(\log n)$.

Graf

Vedno pokaži tabelo:

- trenutna množica;

- kandidati;
- izbrana povezava/vozlišče;
- posodobljene razdalje/barve.

DP

Če ni stanja in rekurence, rešitev ni dovolj dokumentirana.

Minimalno napiši:

1. `dp[...]` pomeni ...
2. Prehod ...
3. Osnova ...
4. Končni odgovor ...

Drevesa

Po vsakem vstavljanju:

1. vstavi kot BST;
2. popravi najnižji neuravnotežen del;
3. nariši stanje.